Latex · the page compiled, in its own folder

1 What sits in the folder, and what a rebuild overwrites

What the folder holds: three files, all built for you, and every one can be built again.

```
<page>/latex/
  <stem>.tex         md2tex's section . header says "do not hand-edit"
  <stem>.pdf         the compiled look . what the view shows
  <stem>-view.html   the tab's page: PDF inline + the .tex one fold below
flat page fallback: <board>/latex/<stem>.* . for a page that owns no folder
```

This part settles which three files live in `latex/`, and that a rebuild may overwrite any of them. Edit one of these files by hand and the next build overwrites it. That is the rule for a folder the machine rebuilds, not a fault. Nobody keeps the view page up to date by hand. `export.py` writes it again at the end of every run, whether or not xelatex produced a PDF. The bibliography looks first at the page's own bibex/ list (QPf8). When `bibex/<stem>.bib` holds an entry, the PDF cites exactly what the page cites. With no page bib, the export looks for a `0-*.bib` by walking up from the page toward the server's root folder. Outside any paper it still compiles with no citations, printing each `\citep` as `[key]` rather than refusing. With either bib the master gains natbib, `plainnat`, and a `bibtex` pass.

2 What survives the trip to LaTeX, and what is lost

What md2tex keeps and what it drops on a board page: most of it lines up, and the losses are written down, not hidden.

```
### 1 . title      -->  \section
#### block         -->  paragraph break
\citep{} \ref{}    -->  kept verbatim
> lanes            -->  DROPPED
refuse-to-regress  -->  inherited
```

This part says what a board page loses on its way to LaTeX: the `>` lanes, and some emoji. The Content parts map cleanly, because a part already is a section. `\citep{}` and `\ref{}` come through untouched, because they were LaTeX before the export started. The `>` lanes are the one real loss. The Word export lands them as comments pinned to a spot, and a LaTeX section has nowhere to put them. The writer also refuses to go backwards: a rewrite that loses citations is refused, not written quietly. The proof run is `QPf4b`'s Content: ten parts became a real nine-page PDF, driven in a browser through the tab. The board's emoji-heavy ascii figures do not survive. xelatex drops any glyph its fonts lack, so a page full of figures reads thinner in PDF than on screen. A2 below is where that gap gets measured instead of guessed.

3 The tab shows something even when the build fails

The tab: the registry's `tab: {url, write}` spec, url first, then a HEAD check, then a build, with the latex route behind it.

```
82-plugin-exports.js --POST--> /_board/latex --runs--> md2tex + xelatex
tab.url()  names latex/<stem>-view.html . PDF framed, .tex one fold below
tab.write() builds on miss . lit-click       > REBUILD (build it again)
no PDF     > the .tex fold opens . log tail above it
```

This part settles that the tab always has a page to show: the PDF when the build works, the log when it does not. One view page is written either way, so the tab is never empty. A run that works shows the PDF, with the raw source folded under it. A run that fails opens that fold and prints the tail of the log, so the error sits where the finished page would have been. Figure 1 shows the working case at the size a reader reads it: page 1 of this page's own export, rendered at 150 dpi, so the claim that the projection works is shown rather than asserted a third time.

Latex · the page compiled, in its own folder

1 What sits in the folder, and what a rebuild overwrites

What the folder holds: three files, all built for you, and every one can be built again.

```
<page>/latex/
  <stem>.tex         md2tex's section . header says "do not hand-edit"
  <stem>.pdf         the compiled look . what the view shows
  <stem>-view.html   the tab's page: PDF inline + the .tex one fold below
flat page fallback: <board>/latex/<stem>.* . for a page that owns no folder
```

This part settles which three files live in `latex/`, and that a rebuild may overwrite any of them. Edit one of these files by hand and the next build overwrites it. That is the rule for a folder the machine rebuilds, not a fault. Nobody keeps the view page up to date by hand. `export.py` writes it again at the end of every run, whether or not xelatex produced a PDF. The bibliography looks first at the page's own bibex/ list (QPf8). When `bibex/<stem>.bib` holds an entry, the PDF cites exactly what the page cites. With no page bib, the export looks for a `0-*.bib` by walking up from the page toward the server's root folder. Outside any paper it still compiles with no citations, printing each `\citep` as `[key]` rather than refusing. With either bib the master gains natbib, `plainnat`, and a `bibtex` pass.

2 What survives the trip to LaTeX, and what is lost

What md2tex keeps and what it drops on a board page: most of it lines up, and the losses are written down, not hidden.

```
### 1 . title      -->  \section
#### block         -->  paragraph break
\citep{} \ref{}    -->  kept verbatim
> lanes            -->  DROPPED
refuse-to-regress  -->  inherited
```

This part says what a board page loses on its way to LaTeX: the `>` lanes, and some emoji. The Content parts map cleanly, because a part already is a section. `\citep{}` and `\ref{}` come through untouched, because they were LaTeX before the export started. The `>` lanes are the one real loss. The Word export lands them as comments pinned to a spot, and a LaTeX section has nowhere to put them. The writer also refuses to go backwards: a rewrite that loses citations is refused, not written quietly. The proof run is `QPf4b`'s Content: ten parts became a real nine-page PDF, driven in a browser through the tab. The board's emoji-heavy ascii figures do not survive. xelatex drops any glyph its fonts lack, so a page full of figures reads thinner in PDF than on screen. A2 below is where that gap gets measured instead of guessed.

3 The tab shows something even when the build fails

The `tab`: the registry's `tab: {url, write}` spec, url first, then a HEAD check, then a build, with the latex route behind it.

1

Figure 1: Page 1 of this page's own LaTeX export, rendered at 150 dpi. The projection is asserted to work in three places on this page and was never shown; this is the showing. A reader who doubts the export can compare this image against `latex/QPf6-latex.pdf` directly.